

# Hands-on 6: Criteria Query

## Problem Statement

Consider an online retail application similar to Amazon.

A user searches for a product using the keyword "**laptop**". The application provides multiple filter options on the left side, such as:

- Customer Review
- Hard Disk Size
- RAM Size
- CPU Speed
- Operating System
- Weight
- CPU

The user may select one or more filters and perform a new search.

The challenge is to dynamically generate the query based on the filters selected by the user.

## Problem with Static HQL Query

In HQL, the query needs to be written with a fixed WHERE clause.

Example:

FROM Product p

WHERE p.name LIKE '%laptop%'

AND p.ram = 16

AND p.hardDisk = 512

AND p.operatingSystem = 'Windows'

However, all users may not select the same filters.

For example:

- User 1 selects only RAM size.
- User 2 selects RAM size and operating system.
- User 3 selects CPU speed, weight, and customer rating.

Creating multiple HQL queries for every possible combination of filters is difficult and not maintainable.

# Dynamic Where Clause Requirement

The WHERE clause should change dynamically based on the selected criteria.

Example:

## Scenario 1:

User selects only RAM size.

Generated query:

```
FROM Product p
```

```
WHERE p.name LIKE '%laptop%'
```

```
AND p.ram = 16
```

## Scenario 2:

User selects RAM size and Operating System.

Generated query:

```
FROM Product p
```

```
WHERE p.name LIKE '%laptop%'
```

```
AND p.ram = 16
```

```
AND p.operatingSystem = 'Windows'
```

## Scenario 3:

User selects CPU Speed, Weight, and Customer Review.

Generated query:

```
FROM Product p
```

```
WHERE p.name LIKE '%laptop%'
```

```
AND p.cpuSpeed >= 3.0
```

```
AND p.weight <= 2
```

```
AND p.rating >= 4
```

# Solution Using Criteria Query

Hibernate Criteria Query provides a programmatic approach to dynamically build queries.

Instead of writing a fixed HQL statement, conditions can be added dynamically depending on the user selection.

The advantages are:

- Dynamic query generation
- Avoids multiple HQL statements
- Easy to add or remove filters
- Improves maintainability

## Criteria Query Implementation Example

```
CriteriaBuilder criteriaBuilder =
```

```
    entityManager.getCriteriaBuilder();
```

```
CriteriaQuery<Product> criteriaQuery =
```

```
    criteriaBuilder.createQuery(Product.class);
```

```
Root<Product> product =
```

```
    criteriaQuery.from(Product.class);
```

```
List<Predicate> predicates = new ArrayList<>();
```

```
// Keyword search
```

```
predicates.add(
```

```
    criteriaBuilder.like(
```

```
        product.get("name"),
```

```
        "%laptop%"
```

```
    )
```

```
);
```

```
// Dynamic RAM filter
```

```
if(ramSize != null) {  
    predicates.add(  
        criteriaBuilder.equal(  
            product.get("ram"),  
            ramSize  
        )  
    );  
}
```

```
// Dynamic Hard Disk filter
```

```
if(hardDiskSize != null) {  
    predicates.add(  
        criteriaBuilder.equal(  
            product.get("hardDisk"),  
            hardDiskSize  
        )  
    );  
}
```

```
// Dynamic Operating System filter
```

```
if(os != null) {  
    predicates.add(  
        criteriaBuilder.equal(  
            product.get("operatingSystem"),
```

```

        os

    )

);

}

// Adding conditions

criteriaQuery.where(

    criteriaBuilder.and(

        predicates.toArray(new Predicate[0])

    )

);

List<Product> products =

    entityManager

        .createQuery(criteriaQuery)

        .getResultList();

```

## Explanation

1. CriteriaBuilder is used to create Criteria queries.
2. CriteriaQuery represents the query structure.
3. Root represents the Product entity.
4. Predicate represents individual filter conditions.
5. Conditions are added only when the user selects the corresponding filter.
6. Hibernate automatically generates the required SQL query.